

Reviewer Pack — Minimal Rank of Coxeter Group Orbits over Tseitin Circuit Va...

Entry 04a45b9fed58 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 18:16:36 UTC

Minimal Rank of Coxeter Group Orbits over Tseitin Circuit Valuations

Entry ID: 04a45b9fed58 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 18:16:36 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry (Coxeter Groups) **Field B** (complexity object): Complexity Theory: Tseitin Circuit Valuations

Statement:

[‘For any given Tseitin circuit valuation instance with n variables, the minimum orbit length in a Coxeter group associated with the circuit is at least exponential in the size of the smallest resolution refutation for that instance.’, ‘Equivalently, for all instances with resolution refutation size $r(n)$, the minimal orbit length in the associated Coxeter group satisfies: $\omega(G) \geq 2^{\Omega(r(n))}$ ’, ‘where $\omega(G)$ is the minimal orbit length in the Coxeter group G .’]

Rationale (proposer’s reasoning):

[‘Coxeter groups are known to be closely related to combinatorial structures that exhibit symmetry, and they have been used in various contexts in algebraic geometry. Their use in complexity theory could potentially reveal new insights into the structure of resolution refutations.’, ‘The conjecture leverages the idea that symmetric structures are more complex to resolve, which aligns with known lower bounds for resolution complexity on expanders.’, ‘This bridge between Coxeter groups and Tseitin circuit valuations could provide a new approach to understanding the inherent complexity of satisfiability problems.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cb7516718355f65f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 24 out of 30 random seeds, the minimal orbit length $\omega(G)$ in the Coxeter group G associated with a Tseitin circuit valuation instance satisfies: $\omega(G) \geq 2^{\Omega(r(n))}$ where $r(n)$ is the resolution refutation size for that instance and Ω denotes the big O notation.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	1.00	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter groups" AND "Tseitin circuit valuations" AND "minimal orbit length" - "resolution refutation size" AND "exponential lower bound" AND "Coxeter group orbits" - "combinatorial geometry" AND Tseitin circuits AND minimal orbit length $\geq 2^{\Omega(r(n))}$

Top relevant hits considered: - [http://arxiv.org/abs/1308.2286v2] A tau-conjecture for Newton polygons - [http://arxiv.org/abs/2209.07583v1] Combinatorial geometry of neural codes, neural data analysis, and neural networks - [http://arxiv.org/abs/2111.05762v1] Non-dimensional Newton-Puiseux Expansions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def tseitin_circuit_valuation(n):
    variables = [f'x{i+1}' for i in range(n)]
    tseitin_vars = [f't{i+1}' for i in range(n)]

    clauses = []
    for i in range(1, n + 1):
        clauses.append([tseitin_vars[2*i-2], -variables[i-1]])
        clauses.append([tseitin_vars[2*i-1], variables[i-1]])
        clauses.append([-tseitin_vars[2*i-2], -tseitin_vars[2*i-1]])

    return variables, clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    variables, clauses = tseitin_circuit_valuation(n)

    # Construct the Coxeter group structure (simplified for demonstration)
    G = {}
    for clause in clauses:
        for lit in clause:
            if lit not in G:
```

```

        G[lit] = set()
    for other_lit in clause:
        if other_lit != lit and -other_lit not in G[lit]:
            G[lit].add(other_lit)

# Compute the minimal orbit length (simplified for demonstration)
visited = set()
def dfs(node):
    if node in visited:
        return 0
    visited.add(node)
    max_depth = 0
    for neighbor in G[node]:
        depth = dfs(neighbor)
        if depth > max_depth:
            max_depth = depth
    return max_depth + 1

min_orbit_length = float('inf')
for node in variables:
    orbit_length = dfs(node)
    if orbit_length < min_orbit_length:
        min_orbit_length = orbit_length

# Compute the resolution refutation size (simplified for demonstration)
r_n = len(clauses)

# Check the conjecture
conjecture_holds = min_orbit_length >= 2 ** math.ceil(math.log2(r_n))
counterexample = "" if conjecture_holds else f"min_orbit_length={min_orbit_length}, expected>=

return {
    "metric_name": "minimal_orbit_length",
    "metric_value": min_orbit_length,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r['metric_value'] for r in results) / len(results)
    std_value = math.sqrt(sum((r['metric_value'] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")

```

```

else:
    first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r['conjecture_hol
    print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\\" first_failing_seed={fi

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a4d5d452.py", line 84, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a4d5d452.py", line 33, in run_trial
    variables, clauses = tseitin_circuit_valuation(n)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a4d5d452.py", line 24, in tseitin_circu
    clauses.append([tseitin_vars[2*i-2], -variables[i-1]])
                  ^^^^^^^^^^^^^^^^^
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture’s support or falsification. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13128
2	preregistration	ollama_remote	glm4:latest	0	0	5834
3	novelty	ollama_remote	glm4:latest	0	0	4899
4	novelty	ollama_remote	glm4:latest	0	0	5704
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44794
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8339
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12452
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10072
9	judge	ollama_remote	glm4:latest	0	0	12084

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117306 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/04a45b9fed58.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/04a45b9fed58.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/04a45b9fed58.tar.gz (if generated)